NestedPropertyEditor - ネスト構造プロパティ編集

1. 概要

1.1 コンポーネントの目的

NestedPropertyEditorは、MAIML XMLファイルのプロパティのネスト構造を階層的に表示・編集するための再帰的コンポーネントです。`propertyListType`の子要素を持つプロパティに対して、深い階層構造の編集を可能にします。

主要機能:

- ネスト構造の階層表示（再帰的レンダリング）
- 展開/折りたたみ制御
- 属性の追加・編集・削除
- 要素の追加・編集・削除
- 子要素の追加（ネスト深化）
- 同階層コピー
- プロパティ削除
- MAIMLスキーマに準拠したプロパティタイプ選択

1.2 使用場面

- PropertyPanelで選択されたプロパティが`propertyListType`の場合
- MethodTabPanelでメソッドのプロパティ編集時
- DocumentEditDialogでドキュメント構造の編集時

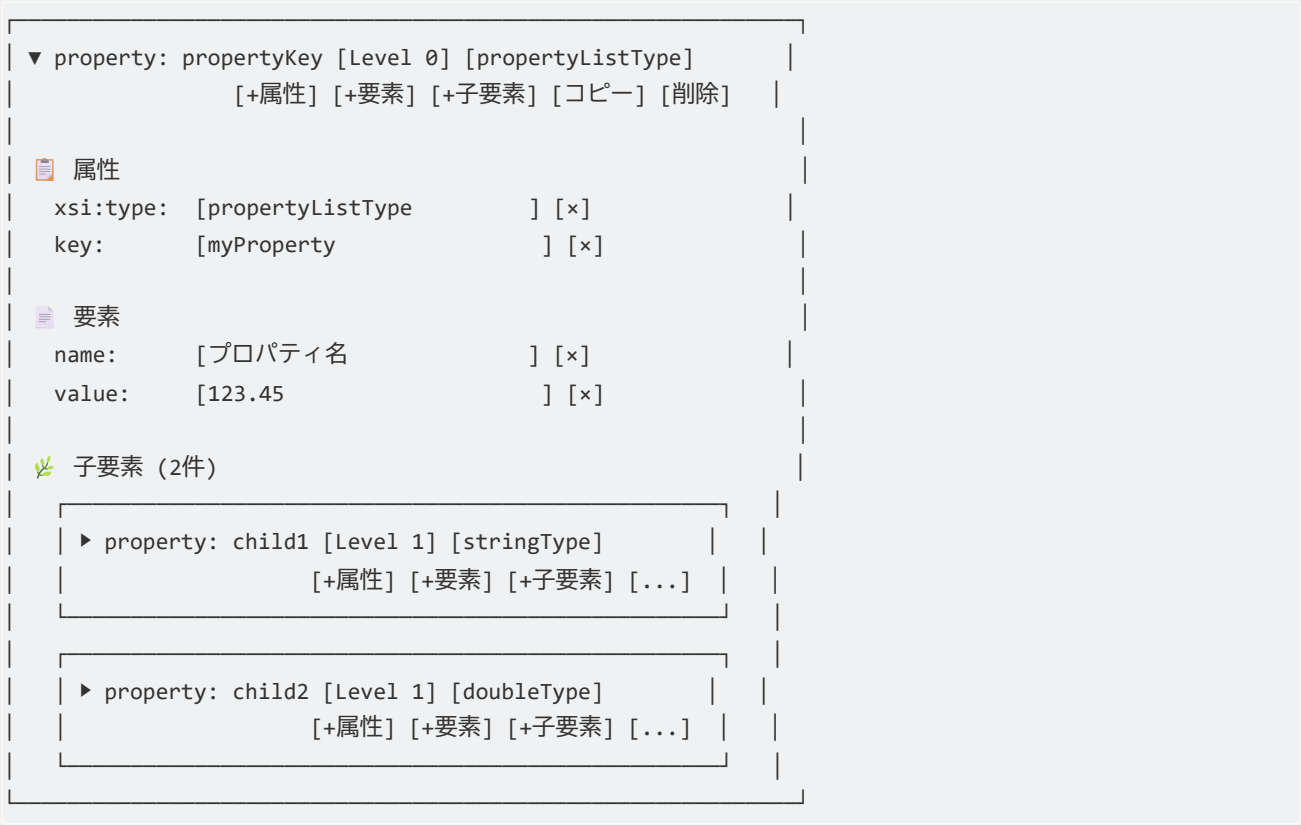
1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造（単一ノード）



2.2 階層構造の表現

2.2.1 インデントレベル

- **Level 0**: インデント0px、青色ボーダー（#1890ff）
- **Level 1**: インデント12px、灰色ボーダー（#d9d9d9）
- **Level 2**: インデント24px、灰色ボーダー
- **Level N**: インデント12N px

2.2.2 背景色の交互配置

- **偶数レベル**: 背景色 #fafafa
- **奇数レベル**: 背景色 #ffffff

2.3 主要UI要素

2.3.1 ヘッダー部分

- **展開/折りたたみボタン**:
  - 表示条件: hasChildren === true
  - アイコン: DownOutlined（展開時）、RightOutlined（折りたたみ時）
  - サイズ: small
  - タイプ: text
- **タイトル**: `{tagName}: {propertyKey}`
  - 例: "property: temperature"
- **Tagラベル**:
  - Level表示: `Level {level}`
  - xsi:type表示:
    - propertyListType: 青色Tag
    - その他: デフォルトTag

2.3.2 操作ボタン群

ボタン	アイコン	タイプ	機能
+属性	PlusOutlined	primary	属性追加モーダルを開く
+要素	PlusOutlined	primary	要素追加モーダルを開く
+子要素	PlusOutlined	primary	子要素追加モーダルを開く
コピー	CopyOutlined	default	同階層コピーモーダルを開く
削除	DeleteOutlined	danger	プロパティ削除（確認ダイアログ）

2.3.3 属性セクション

- **ラベル**: "📋 属性"（success色、12px）
- **表示条件**: hasAttributes === true

- **各属性行**:
  - 属性名（100px固定幅）
  - Input（編集可能、flex: 1）
  - 削除ボタン（CloseOutlined、danger）

2.3.4 要素セクション

- **ラベル**: "📄 要素"（secondary色、12px）
- **表示条件**: hasElements === true
- **各要素行**:
  - 要素名（100px固定幅）
  - Input（編集可能、flex: 1）
  - 削除ボタン（CloseOutlined、danger）

2.3.5 子要素セクション

- **ラベル**: "🌿 子要素 (N件)"（紫色 #722ed1、12px、bold）
- **表示条件**: hasChildren === true && isExpanded === true
- **再帰レンダリング**: 各子要素に対してNestedPropertyEditorを再帰的にレンダリング

---

3. 機能仕様

3.1 Props定義

```
interface NestedPropertyEditorProps {
  propertyData: PropertyData;
  propertyKey: string;
  level?: number;
  expandedNodes: Set<string>;
  onToggleExpanded: (nodeId: string) => void;
  onAddNestedChild: (parentKey: string, childTagName: string, specifiedKey?: string) => void;
  onAddSameLevelCopy: (sourceData: PropertyData, newKey: string) => void;
  onUpdateNestedAttribute: (propertyKey: string, attrName: string, attrValue: string) => void;
  onUpdateNestedElement: (propertyKey: string, elemName: string, elemValue: string) => void;
```

```
  onDeleteNestedAttribute: (propertyKey: string, attrName: string) => void;
  onDeleteNestedElement: (propertyKey: string, elemName: string) => void;
  onDeleteProperty: (propertyKey: string) => void;
  forceRenderKey: number;
}

interface PropertyData {
  key: string;
  tagName: string;
  attributes: Record<string, string>;
  elements: Record<string, string>;
  children: PropertyData[];
  namespaceOrigin?: boolean;
  namespaceSource?: NamespaceSource;
}

interface NamespaceSource {
  fileName?: string;
  uri?: string;
  hierarchyPath?: string;
  importedAt?: string;
}
```

Props説明:

Prop名	型	必須	説明
propertyData	PropertyData	✅	プロパティデータ
propertyKey	string	✅	プロパティのキー
level	number	-	ネスト階層レベル（デフォルト: 0）
expandedNodes	Set	✅	展開状態のノードID Set
onToggleExpanded	(nodeId) => void	✅	展開/折りたたみトグルコールバック
onAddNestedChild	(parentKey, childTagName, key?) => void	✅	子要素追加コールバック
onAddSameLevelCopy	(sourceData, newKey) => void	✅	同階層コピーコールバック
onUpdateNestedAttribute	(propertyKey, attrName, attrValue) => void	✅	属性更新コールバック
onUpdateNestedElement	(propertyKey, elemName, elemValue) => void	✅	要素更新コールバック
onDeleteNestedAttribute	(propertyKey, attrName) => void	✅	属性削除コールバック
onDeleteNestedElement	(propertyKey, elemName) => void	✅	要素削除コールバック
onDeleteProperty	(propertyKey) => void	✅	プロパティ削除コールバック
forceRenderKey	number	✅	強制再レンダリング用キー

3.2 State定義

```
const [showAttributeAddModal, setShowAttributeAddModal] = useState(false);
const [showElementAddModal, setShowElementAddModal] = useState(false);
const [showChildAddModal, setShowChildAddModal] = useState(false);
const [showCopyModal, setShowCopyModal] = useState(false);
const [newAttributeName, setNewAttributeName] = useState('');
const [newAttributeValue, setNewAttributeValue] = useState('');
const [newElementName, setNewElementName] = useState('');
const [newElementValue, setNewElementValue] = useState('');
const [newChildTagName, setNewChildTagName] = useState('property');
const [copyKey, setCopyKey] = useState('');
const [customAttributeName, setCustomAttributeName] = useState('');
```

State説明:

State名	型	初期値	説明
showAttributeAddModal	boolean	false	属性追加モーダル表示フラグ
showElementAddModal	boolean	false	要素追加モーダル表示フラグ
showChildAddModal	boolean	false	子要素追加モーダル表示フラグ
showCopyModal	boolean	false	同階層コピーモーダル表示フラグ
newAttributeName	string	''	新規属性名
newAttributeValue	string	''	新規属性値
newElementName	string	''	新規要素名
newElementValue	string	''	新規要素値
newChildTagName	string	'property'	新規子要素のタグ名
copyKey	string	''	コピー時の新しいキー
customAttributeName	string	''	カスタム属性名

3.3 計算値（useMemo）

3.3.1 nodeId

```
const nodeId = useMemo(() => propertyKey, [propertyKey]);
```

3.3.2 isExpanded

```
const isExpanded = useMemo(() => expandedNodes.has(nodeId), [expandedNodes, nodeId]);
```

3.3.3 hasChildren

```
const hasChildren = useMemo(() => {
  const children = propertyData.children;
  const parentXsiType = propertyData.attributes?.['xsi:type'] || '';
  const isParentPropertyListType = parentXsiType === 'propertyListType';
  const isValidArray = children && Array.isArray(children);
  const hasLength = isValidArray && children.length > 0;
  const hasValidChildren = hasLength && children.some(child =>
    child && typeof child === 'object' && (child.key || child.tagName)
  );

  return isParentPropertyListType && hasValidChildren;
}, [propertyData.children, propertyData.attributes]);
```

判定条件:

- xsi:type が `propertyListType`
- children が有効な配列
- children に有効な子要素が存在

3.3.4 hasAttributes

```
const hasAttributes = useMemo(() =>
  propertyData.attributes && Object.keys(propertyData.attributes).length > 0,
  [propertyData.attributes]
);
```

3.3.5 hasElements

```
const hasElements = useMemo(() =>
  propertyData.elements && Object.keys(propertyData.elements).length > 0,
  [propertyData.elements]
);
```

3.4 展開/折りたたみ

3.4.1 処理フロー

```
<Button
  icon={isExpanded ? <DownOutlined /> : <RightOutlined />}
  onClick={() => onToggleExpanded(nodeId)}
/>
```

3.4.2 UI動作

- ユーザー操作: 展開/折りたたみボタンをクリック
- 処理: 親コンポーネントのonToggleExpanded実行
- 結果: expandedNodesが更新され、子要素セクションの表示/非表示が切り替わる

3.5 属性追加

3.5.1 処理フロー

```
// モーダルを開く
<Button onClick={() => setShowAttributeAddModal(true)}>+属性</Button>

// 確定処理
onOk={() => {
  const finalAttributeName = newAttributeName === '__custom__'
    ? customAttributeName.trim()
    : newAttributeName.trim();

  if (!finalAttributeName) {
    message.error('属性名を入力してください');
    return;
  }
```

```
  if (!newAttributeValue.trim()) {
    message.error('属性値を入力してください');
    return;
  }

  onUpdateNestedAttribute(propertyKey, finalAttributeName, newAttributeValue.trim());
  // リセット
  setNewAttributeName('');
  setNewAttributeValue('');
  setCustomAttributeName('');
  setShowAttributeAddModal(false);
}}
```

3.5.2 属性名の選択肢

属性名	説明
xsi:type	MaiML型定義（専用セレクト表示）
units	単位
formatString	書式文字列
scaleFactor	スケール係数
key	キー
id	ID
ref	参照
custom	カスタム属性名入力

3.5.3 xsi:type選択時の特殊処理

- **デフォルト値**: 'stringType'を自動設定
- **専用セレクト表示**: MAIMLプロパティタイプ一覧から選択

3.6 MAIMLプロパティタイプ一覧

3.6.1 Simple Types（シンプル型）

- stringType, tokenType, idType, idRefType
- qualifiedNameType, dateTimeType
- decimalType, doubleType, floatType

- intType, longType, shortType, byteType
- unsignedIntType, unsignedLongType, unsignedShortType, unsignedByteType
- booleanType
- base64BinaryType, hexBinaryType
- uriType, uuidType, languageType

3.6.2 List Types（リスト型）

- **propertyListType**: プロパティリスト（子要素を持つ）
- stringListType, idRefListType, qualifiedNameListType
- dateTimeListType
- decimalListType, doubleListType, floatListType
- intListType, longListType, shortListType, byteListType
- unsignedIntListType, unsignedLongListType, unsignedShortListType, unsignedByteListType
- booleanListType
- base64BinaryListType, hexBinaryListType
- uriListType, uuidListType, languageListType
- stringEnumType

3.7 要素追加

3.7.1 処理フロー

```
onOk={() => {
  if (newElementName.trim() && newElementValue.trim()) {
    onUpdateNestedElement(propertyKey, newElementName.trim(), newElementValue.trim());
    setNewElementName('');
    setNewElementValue('');
    setShowElementAddModal(false);
  }
}}
```

3.7.2 入力フィールド

- **要素名**: Input（プレースホルダー: "value, description"）

- **要素値**: TextArea（3行）

3.8 子要素追加

3.8.1 処理フロー

```
onOk={() => {
  if (copyKey.trim() && newChildTagName.trim()) {
    onAddNestedChild(propertyKey, newChildTagName.trim(), copyKey.trim());
    setCopyKey('');
    setNewChildTagName('property');
    setShowChildAddModal(false);
  }
}}
```

3.8.2 タグ名の選択肢

- **property**: プロパティ要素（デフォルト）
- **content**: コンテンツ要素
- **parameter**: パラメータ要素

3.8.3 情報表示Alert

- **タイトル**: `親要素: {propertyKey} | 現在の子要素数: {N}件`
- **タイプ**: info

3.9 同階層コピー

3.9.1 処理フロー

```
onOk={() => {
  if (copyKey.trim()) {
    onAddSameLevelCopy(propertyData, copyKey.trim());
    setCopyKey('');
    setShowCopyModal(false);
```

```
  }
}}
```

3.9.2 情報表示Alert

- **タイトル**: `コピー元: {propertyKey}`
- **タイプ**: info

3.10 プロパティ削除

3.10.1 処理フロー

```
onClick={() => {
  modal.confirm({
    title: `プロパティ「${propertyKey}」を削除しますか？`,
    onOk: () => onDeleteProperty(propertyKey),
  });
}}
```

3.10.2 UI動作

- ユーザー操作: 削除ボタンをクリック
- 確認ダイアログ表示
- OK → onDeleteProperty実行
- キャンセル → 処理中止

3.11 属性・要素の編集/削除

3.11.1 属性編集

```
<Input
  value={attrValue || ''}
  onChange={(e) => onUpdateNestedAttribute(propertyKey, attrName, e.target.value)}
/>
```

3.11.2 属性削除

```
<Button
  icon={<CloseOutlined />}
  onClick={() => onDeleteNestedAttribute(propertyKey, attrName)}
/>
```

3.11.3 要素編集・削除

- 属性と同様の実装パターン

3.12 再帰レンダリング

3.12.1 処理フロー

```
{hasChildren && isExpanded && (
  <div>
    {propertyData.children.map((child, index) => {
      const childKey = child.key || `${child.tagName || 'child'}_${index}`;
      return (
        <NestedPropertyEditor
          key={`${childKey}_${index}_${forceRenderKey}`}
          propertyData={child}
          propertyKey={childKey}
          level={level + 1}
          expandedNodes={expandedNodes}
          // ... その他のProps
          forceRenderKey={forceRenderKey}
        />
      );
    })}
  </div>
)}
```

3.12.2 キー生成

- **ユニークキー**: `{childKey}_{index}_{forceRenderKey}`
- **forceRenderKey**: 親コンポーネントから渡され、強制再レンダリングを制御

---

4. スタイル仕様

4.1 コンテナスタイル

```
<div style={{
  marginLeft: `${level * 12}px`,
  marginBottom: 4
}}>
  <div style={{
    border: `2px solid ${level === 0 ? '#1890ff' : '#d9d9d9'}`,
    borderRadius: 8,
    padding: 12,
    backgroundColor: level % 2 === 0 ? '#fafafa' : '#ffffff'
  }}>
```

説明:

- **インデント**: level * 12px
- **ボーダー**: Level 0は青色、それ以外は灰色
- **角丸**: 8px
- **パディング**: 12px
- **背景色**: 偶数レベルは#fafafa、奇数レベルは#ffffff

4.2 属性・要素行のスタイル

```
<div style={{
  display: 'flex',
  alignItems: 'center',
  gap: 8,
  marginBottom: 4
}}>
  <Text style={{ minWidth: 100, fontSize: 12 }}>{attrName}:</Text>
  <Input style={{ flex: 1 }} />
  <Button />
</div>
```

説明:

- **レイアウト**: フレックスボックス、水平配置
- **ギャップ**: 8px
- **属性名**: 最小幅100px、フォントサイズ12px
- **Input**: flex: 1（残りスペースを占有）

4.3 セクションラベル

```
// 属性セクション
<Text type="success" strong style={{ fontSize: 12 }}>📋 属性</Text>

// 要素セクション
<Text type="secondary" strong style={{ fontSize: 12 }}>📄 要素</Text>

// 子要素セクション
<Text style={{ fontSize: 12, color: '#722ed1', fontWeight: 'bold' }}>
  🌿 子要素 ({N}件)
</Text>
```

---

5. バリデーション

5.1 属性追加時のバリデーション

条件	エラーメッセージ
属性名が空	"属性名を入力してください"
属性値が空	"属性値を入力してください"

5.2 要素追加時のバリデーション

- 要素名と要素値の両方が必須（空の場合は処理をスキップ）

5.3 子要素追加時のバリデーション

- キー名とタグ名の両方が必須（空の場合は処理をスキップ）

5.4 同階層コピー時のバリデーション

- 新しいキー名が必須（空の場合は処理をスキップ）

---

6. エラーハンドリング

6.1 エラーケース

エラーケース	対処方法
属性名が空	message.error表示、処理中止
属性値が空	message.error表示、処理中止
子要素のキー重複	親コンポーネントで処理

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/NestedPropertyEditor.tsx`

7.2 依存コンポーネント

- Ant Design:
  - Button, Input, Alert, Space, Typography, Select, Tag, message, Modal, App
- Ant Design Icons:
  - CloseOutlined, DownOutlined, RightOutlined, PlusOutlined, DeleteOutlined, CopyOutlined

7.3 親コンポーネント

- PropertyPanel
- MethodTabPanel
- DocumentEditDialog

7.4 型定義

- PropertyData
- NamespaceSource
- NestedPropertyEditorProps

---

8. 使用例

8.1 基本的な使用

```
<NestedPropertyEditor
  propertyData={propertyData}
  propertyKey="temperature"
  level={0}
  expandedNodes={expandedNodes}
  onToggleExpanded={handleToggleExpanded}
  onAddNestedChild={handleAddNestedChild}
  onAddSameLevelCopy={handleAddSameLevelCopy}
  onUpdateNestedAttribute={handleUpdateNestedAttribute}
  onUpdateNestedElement={handleUpdateNestedElement}
  onDeleteNestedAttribute={handleDeleteNestedAttribute}
  onDeleteNestedElement={handleDeleteNestedElement}
  onDeleteProperty={handleDeleteProperty}
  forceRenderKey={forceRenderKey}
/>
```

8.2 親コンポーネントでのコールバック実装例

```
// 展開/折りたたみトグル
const handleToggleExpanded = (nodeId: string) => {
  setExpandedNodes(prev => {
    const newSet = new Set(prev);
    if (newSet.has(nodeId)) {
```

```
      newSet.delete(nodeId);
    } else {
      newSet.add(nodeId);
    }
    return newSet;
  });
};

// 属性更新
const handleUpdateNestedAttribute = (
  propertyKey: string,
  attrName: string,
  attrValue: string
) => {
  // 再帰的に該当プロパティを探して属性を更新
  const updatedProperties = updateNestedProperty(
    properties,
    propertyKey,
    (prop) => ({
      ...prop,
      attributes: {
        ...prop.attributes,
        [attrName]: attrValue
      }
    })
  );
  setProperties(updatedProperties);
  setUnsavedChanges(true);
};

// 子要素追加
const handleAddNestedChild = (
  parentKey: string,
  childTagName: string,
  specifiedKey?: string
) => {
  const newChild: PropertyData = {
    key: specifiedKey || `newProperty_${Date.now()}`,
    tagName: childTagName,
    attributes: { 'xsi:type': 'stringType' },
    elements: {},
    children: []
  };

  const updatedProperties = updateNestedProperty(
    properties,
    parentKey,
    (prop) => ({
      ...prop,
      children: [...(prop.children || []), newChild]
    })
  );
```

```
  setProperties(updatedProperties);
  setForceRenderKey(prev => prev + 1); // 強制再レンダリング
  setUnsavedChanges(true);
};
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-02-13	カスタム属性名対応（Phase 2-2）、v2.0.0	Development Team
2026-01-29	TemplateEditorDialogから分離、独立版作成	Development Team